

MedAssist-AI

Disease Prediction Model Development, Optimization, and System Integration
Sanvi Sawant

1. Project Description

Milestone 4 focused on deploying the completed MedAssist AI application and making the system accessible through a live web environment. The work involved preparing the production frontend and backend, configuring deployment environments, deploying the React frontend on Vercel and the FastAPI backend on Render, connecting the deployed services, configuring environment variables and validating the complete application in its hosted environment.

2. Application Used for Deployment

The final MedAssist AI application developed during the previous milestones was used as the deployment candidate. The application consists of a React 19 + Vite frontend, FastAPI backend, integrated machine learning prediction pipeline, database layer, authentication, analytics, reporting and recommendation modules. The trained Voting Classifier and its associated preprocessing and disease-mapping components were included in the production workflow.

3. Environment Setup

The deployment environment consisted of React 19, Vite, FastAPI, Python 3.10+, Scikit-learn, XGBoost, LightGBM, Pandas, NumPy, Joblib, PostgreSQL/SQLite, JWT authentication, GitHub, Vercel and Render. Environment-specific configuration was maintained using deployment environment variables rather than exposing local configuration values in the repository.

4. Pre-Deployment Preparation

Before deployment, the application was prepared for production execution. The frontend and backend were verified independently, required dependencies were checked, production configuration files were reviewed, API communication settings were updated for the hosted environment, and the machine learning model-loading process was verified. The repository structure was also organized so that the production frontend and backend could be deployed independently while continuing to work as one application.

5. Backend Deployment on Render

The FastAPI backend was deployed using Render. The backend repository configuration was prepared with the required Python dependencies and deployment settings. Environment variables were configured for application settings, authentication, database connectivity and other runtime requirements. The deployed backend provides the REST API used by the frontend for authentication, symptom retrieval, disease prediction, risk assessment, recommendations, reports and analytics. The machine learning pipeline was configured to load the required model and supporting resources in the hosted environment.

6. Frontend Deployment on Vercel

The React 19 + Vite frontend was deployed using Vercel. The frontend build configuration and required environment variables were configured so that the application communicates with the deployed FastAPI backend instead of the local development server. The production build was tested to verify routing, authentication, symptom submission, prediction results, dashboards, reports and other frontend workflows.

7. Frontend-Backend Connection

After deploying both components, the Vercel frontend was connected to the Render backend through the configured API base URL. Cross-origin communication and production environment configuration were verified so that requests from the hosted frontend could reach the backend successfully. The complete request flow was validated from symptom submission in the frontend, through the FastAPI API and machine learning prediction pipeline, to the returned prediction, risk assessment and application results.

8. Machine Learning Deployment

The trained Voting Classifier was incorporated into the deployed backend prediction pipeline. The deployment configuration ensured that the required model artifacts, preprocessing logic and disease-name mapping were available during inference. The model-loading mechanism was also configured to support the cloud deployment environment, including remote model retrieval where required for lightweight deployment. Prediction generation was tested after deployment to confirm that the hosted backend produced results consistently with the locally tested application.

9. Database and Authentication Configuration

The deployed application retained the authentication and database functionality implemented during the previous milestone. JWT-based authentication was configured using deployment environment variables, while the database configuration was adapted for the hosted environment. Patient and doctor access flows were verified to ensure that protected routes, user-specific information, prediction history and application dashboards remained functional after deployment.

10. Deployment Validation and Testing

The deployed system was validated through end-to-end testing. The frontend hosted on Vercel was tested against the backend hosted on Render, including user registration and login, symptom selection, disease prediction, risk assessment, recommendations, report generation and analytics. API responses, model loading, frontend routing, authentication, database operations and production build execution were checked to identify deployment-specific issues. The final live workflow was verified from user interaction through backend processing and machine learning inference to result presentation.

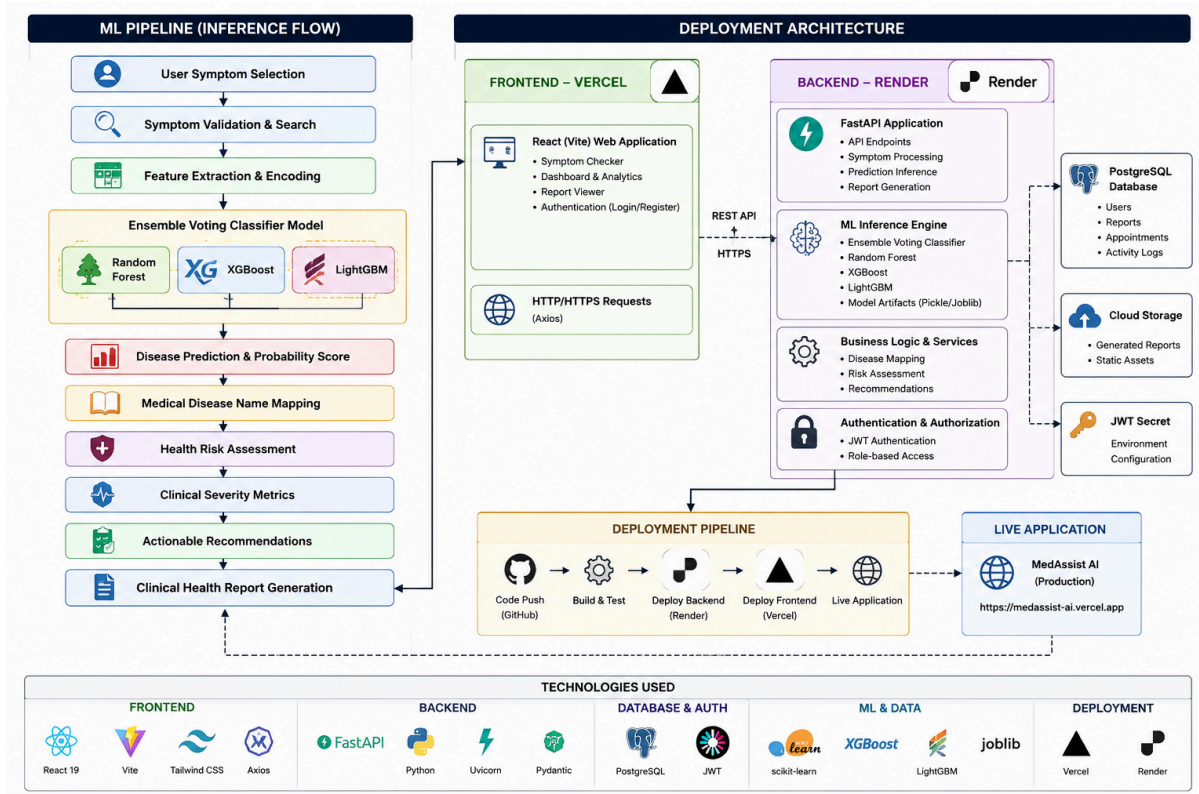


Figure 1: MedAssist AI System Deployment and Machine Learning Prediction Workflow

11. Tasks Completed

- Production preparation of the completed MedAssist AI application
- Configuration and deployment of the FastAPI backend on Render
- Configuration and deployment of the React/Vite frontend on Vercel
- Connecting the hosted frontend with the hosted backend
- Configuration of production environment variables
- Integration and verification of the deployed machine learning prediction pipeline
- Validation of authentication, database, prediction, reporting and analytics workflows
- End-to-end testing of the live application

12. Challenges

The major deployment challenges included configuring separate frontend and backend environments, handling production environment variables, connecting the Vercel frontend with the Render API, ensuring correct model loading in a cloud environment, maintaining disease mapping and preprocessing during remote inference, and resolving differences between local and hosted execution. Additional validation was required to ensure that authentication, database operations and API communication remained stable after deployment.

13. Conclusion

Milestone 4 completed the deployment phase of MedAssist AI by making the application available through a hosted frontend and backend architecture. The React frontend was deployed on Vercel and the FastAPI backend on Render, with the machine learning pipeline, authentication, database, reporting, analytics and prediction workflows connected and validated in the production environment. The final deployment transformed the previously integrated application into an accessible live healthcare platform.